CREATED AND MODIFIED CODE

Main.cpp

```
    void initialise (const String& /*commandLine*/) override
    {
        // This method is where you should put your application's initialisation
code.
        // Creating the main window and setting the app name
        mainWindow.reset (new MainWindow (getApplicationName()));
    }

    void shutdown() override
    {

        // Cleaning up the main window to close the app properly
        mainWindow = nullptr; // (deletes our window)
    }

    void anotherInstanceStarted (const String& /*commandLine*/) override
    {
        // When another instance of the app is launched while this one is running,
        // this method is invoked, and the commandLine parameter tells you what
        // the other instance's command-line arguments were.
    }

    class MainWindow    : public DocumentWindow
    {
    public:
        MainWindow (String name)  : DocumentWindow (name,

Desktop::getInstance().getDefaultLookAndFeel()

.findColour (ResizableWindow::backgroundColourId),
                                                    DocumentWindow::allButtons)
        {
            setUsingNativeTitleBar (true);

            // Adding our MainComponent to the window
            setContentOwned (new MainComponent(), true);

           #if JUCE_IOS || JUCE_ANDROID
            setFullScreen (true);
           #else
            // Setting the window to be resizable and centring it on the screen
            setResizable (true, true);
            centreWithSize (getWidth(), getHeight());
           #endif

            // Making the window visible to the user
```

```
            setVisible (true);
        }

        void closeButtonPressed() override
        {
            // This is called when the user tries to close this window. Here,
we'll just
            // ask the app to quit when this happens, but you can change this to
do
            // whatever you need.
            JUCEApplication::getInstance()->systemRequestedQuit();
        }


    private:
        JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (MainWindow)
    };

private:
    // Smart pointer to manage the main window's lifetime
    std::unique_ptr<MainWindow> mainWindow;
```

DeckGUI.h

```
class DeckGUI    : public Component,
                   public Button::Listener,
                   public Slider::Listener,
                   public FileDragAndDropTarget,
                   public Timer
{
public:
    /** Constructor: sets up all the buttons, sliders, and labels, and starts the
UI timer. */
    DeckGUI(DJAudioPlayer* player,
           AudioFormatManager &    formatManagerToUse,
           AudioThumbnailCache &   cacheToUse );

    ~DeckGUI();

    void paint (Graphics& g) override;
    void resized() override;

    /** Loads audio file directly from deck or playlist. */
    void loadFile(juce::URL audioURL);

    /** Sets the main theme colour for the deck. */
    void setDeckColor(juce::Colour color);

    void buttonClicked (Button *) override;
    void sliderValueChanged (Slider *slider) override;
```

```
    bool isInterestedInFileDrag (const StringArray &files) override;
    void filesDropped (const StringArray &files, int x, int y) override;

    void timerCallback() override;

private:
    void saveCues();
    void loadCues(juce::URL trackURL);
    void saveEQ();
    void loadEQ(juce::URL trackURL);

    juce::TextButton resetEQButton{"RESET EQ"};
    juce::FileChooser fChooser{"Select a file..."};

    TextButton playButton{"PLAY"};
    TextButton stopButton{"STOP"};
    TextButton loadButton{"LOAD"};

    juce::TextButton cueButtons[8];
    juce::TextButton clearCuesButton{"Clear"};
    double cuePositions[8];

    juce::URL currentURL;

    Slider volSlider;
    Slider speedSlider;
    Slider posSlider;

    WaveformDisplay waveformDisplay;
    DJAudioPlayer* player;

    juce::Slider eqLowSlider;
    juce::Slider eqMidSlider;
    juce::Slider eqHighSlider;

    juce::Label volLabel{ "VOL", "VOL" };
    juce::Label speedLabel{ "SPD", "SPD" };
    juce::Label posLabel{ "POS", "POS" };
    juce::Label eqLowLabel{ "LOW", "LOW" };
    juce::Label eqMidLabel{ "MID", "MID" };
    juce::Label eqHighLabel{ "HIGH", "HIGH" };

    juce::Colour deckColor;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (DeckGUI)
};
```

DeckGUI.cpp

```
DeckGUI::DeckGUI(DJAudioPlayer* _player,
                 AudioFormatManager &    formatManagerToUse,
```

```
                  AudioThumbnailCache &    cacheToUse
              ) : player(_player),
                  waveformDisplay(formatManagerToUse, cacheToUse)
{
    addAndMakeVisible(playButton);
    addAndMakeVisible(stopButton);
    addAndMakeVisible(loadButton);
    addAndMakeVisible(volSlider);
    addAndMakeVisible(speedSlider);
    addAndMakeVisible(posSlider);
    addAndMakeVisible(waveformDisplay);

    addAndMakeVisible(volLabel);
    addAndMakeVisible(speedLabel);
    addAndMakeVisible(posLabel);

    volLabel.setJustificationType(juce::Justification::centredLeft);
    speedLabel.setJustificationType(juce::Justification::centredLeft);
    posLabel.setJustificationType(juce::Justification::centredLeft);

    addAndMakeVisible(eqLowSlider);
    addAndMakeVisible(eqMidSlider);
    addAndMakeVisible(eqHighSlider);
    addAndMakeVisible(eqLowLabel);
    addAndMakeVisible(eqMidLabel);
    addAndMakeVisible(eqHighLabel);
    addAndMakeVisible(resetEQButton);

    for (int i = 0; i < 8; ++i) {
        cueButtons[i].setButtonText(std::to_string(i + 1));
        addAndMakeVisible(cueButtons[i]);
        cueButtons[i].addListener(this);
        cuePositions[i] = -1.0;
    }
    addAndMakeVisible(clearCuesButton);

    playButton.addListener(this);
    stopButton.addListener(this);
    loadButton.addListener(this);
    volSlider.addListener(this);
    speedSlider.addListener(this);
    posSlider.addListener(this);
    eqLowSlider.addListener(this);
    eqMidSlider.addListener(this);
    eqHighSlider.addListener(this);
    resetEQButton.addListener(this);
    clearCuesButton.addListener(this);

    volSlider.setRange(0.0, 1.0);
    speedSlider.setRange(0.0, 10.0);
    posSlider.setRange(0.0, 1.0);
    eqLowSlider.setRange(0.0, 2.0);
    eqMidSlider.setRange(0.0, 2.0);
    eqHighSlider.setRange(0.0, 2.0);
```

```
    startTimer(500);
}

void DeckGUI::resized()
{
    double rowH = getHeight() / 9;
    double colW = getWidth() / 4;

    playButton.setBounds(0, 0, colW * 2, rowH);
    stopButton.setBounds(colW * 2, 0, colW * 2, rowH);

    volLabel.setBounds(5, rowH, 40, rowH);
    volSlider.setBounds(45, rowH, getWidth() - 50, rowH);

    speedLabel.setBounds(5, rowH * 2, 40, rowH);
    speedSlider.setBounds(45, rowH * 2, getWidth() - 50, rowH);

    posLabel.setBounds(5, rowH * 3, 40, rowH);
    posSlider.setBounds(45, rowH * 3, getWidth() - 50, rowH);

    double eqW = getWidth() / 3;
    eqLowSlider.setBounds(0, rowH * 4, eqW, rowH * 1.5);
    eqMidSlider.setBounds(eqW, rowH * 4, eqW, rowH * 1.5);
    eqHighSlider.setBounds(eqW * 2, rowH * 4, eqW, rowH * 1.5);

    double cueW = getWidth() / 5;
    for (int i = 0; i < 4; ++i) {
        cueButtons[i].setBounds(i * cueW, rowH * 5.5, cueW, rowH);
        cueButtons[i + 4].setBounds(i * cueW, rowH * 6.5, cueW, rowH);
    }
    clearCuesButton.setBounds(4 * cueW, rowH * 5.5, cueW, rowH * 2);

    waveformDisplay.setBounds(0, rowH * 7.5, getWidth(), rowH);
    loadButton.setBounds(0, rowH * 8.5, getWidth(), rowH / 2);
}

void DeckGUI::buttonClicked(Button* button)
{
    if (button == &playButton) player->start();
    if (button == &stopButton) player->stop();
    if (button == &loadButton)
    {
        auto fileChooserFlags = FileBrowserComponent::canSelectFiles;
        fChooser.launchAsync(fileChooserFlags, [this](const FileChooser& chooser)
        {
            File chosenFile = chooser.getResult();
            if (chosenFile.exists()) {
                loadFile(URL{chosenFile});
            }
        });
    }

    if (button == &resetEQButton) {
```

```
        eqLowSlider.setValue(1.0);
        eqMidSlider.setValue(1.0);
        eqHighSlider.setValue(1.0);
    }

    if (button == &clearCuesButton) {
        for (int i = 0; i < 8; ++i) {
            cuePositions[i] = -1.0;
            cueButtons[i].setColour(TextButton::buttonColourId,
Colours::darkgrey);
        }
        saveCues();
    }

    for (int i = 0; i < 8; ++i) {
        if (button == &cueButtons[i]) {
            if (cuePositions[i] >= 0) {
                player->setPositionRelative(cuePositions[i]);
            } else {
                cuePositions[i] = player->getPositionRelative();
                cueButtons[i].setColour(TextButton::buttonColourId, Colours::red);
                saveCues();
            }
        }
    }
}
```

DJAudioPlayer.h

```
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"
#include <juce_dsp/juce_dsp.h>

class DJAudioPlayer : public AudioSource {
  public:

    /** Constructor: initializes the player and sets up the audio format manager.
     * @param _formatManager The manager that helps the player recognize different
audio files.
     */
    DJAudioPlayer(AudioFormatManager& _formatManager);

    /** Destructor: cleans up the resources when the player is no longer needed.
*/
    ~DJAudioPlayer();

    /** Prepares the audio source for playback.
     * @param samplesPerBlockExpected The number of samples the player should
process at a time.
     * @param sampleRate The sample rate of the audio output device.
```

```
     */
    void prepareToPlay (int samplesPerBlockExpected, double sampleRate) override;

    /** Fills the audio buffer with the next block of data to be played.
     * @param bufferToFill Information about the buffer that needs audio data.
     */
    void getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill) override;

    /** Releases audio resources when playback is finished or stopped. */
    void releaseResources() override;

    /** Loads a specific audio file from a URL.
     * @param audioURL The location of the audio file to load.
     */
    void loadURL(URL audioURL);

    /** Adjusts the volume gain of the player.
     * @param gain The volume level (0.0 to 1.0).
     */
    void setGain(double gain);

    /** Adjusts the playback speed ratio.
     * @param ratio The speed multiplier (e.g., 1.0 is normal speed).
     */
    void setSpeed(double ratio);

    /** Sets the playback position in seconds.
     * @param posInSecs The time in seconds to jump to.
     */
    void setPosition(double posInSecs);

    /** Sets the playback position as a fraction of the total duration.
     * @param pos The percentage of the track to jump to.
     */
    void setPositionRelative(double pos);

    /** Adjusts the low-frequency gain of the equaliser.
     * @param gainLinear The gain level for the low band.
     */
    void setEqLow(float gainLinear);

    /** Adjusts the mid-frequency gain of the equaliser.
     * @param gainLinear The gain level for the mid band.
     */
    void setEqMid(float gainLinear);

    /** Adjusts the high-frequency gain of the equaliser.
     * @param gainLinear The gain level for the high band.
     */
    void setEqHigh(float gainLinear);

    /** Starts the audio playback. */
    void start();
```

```
    /** Stops the audio playback. */
    void stop();

    /** get the relative position of the playhead */
    double getPositionRelative();

    /** Calculates the current BPM based on the track and playback speed.
     * @return The calculated beats per minute.
     */
    double getBpm();

private:
    AudioFormatManager& formatManager;
    std::unique_ptr<AudioFormatReaderSource> readerSource;
    AudioTransportSource transportSource;
    ResamplingAudioSource resampleSource{&transportSource, false, 2};
    double currentSampleRate = 44100.0;

    juce::dsp::ProcessorChain<juce::dsp::IIR::Filter<float>,
                              juce::dsp::IIR::Filter<float>,
                              juce::dsp::IIR::Filter<float>> eqChain;

};
```

DJAudioPlayer.cpp

```
#include "DJAudioPlayer.h"

// The constructor: links the format manager to our player
DJAudioPlayer::DJAudioPlayer(AudioFormatManager& _formatManager)
: formatManager(_formatManager)
{

}

// The destructor: cleans up when the player is destroyed
DJAudioPlayer::~DJAudioPlayer()
{

}

// Prepare our sources and the EQ chain for audio playback
void DJAudioPlayer::prepareToPlay (int samplesPerBlockExpected, double sampleRate)
{
    transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
    resampleSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

    currentSampleRate = sampleRate;

    juce::dsp::ProcessSpec spec;
    spec.sampleRate = sampleRate;
```

```
    spec.maximumBlockSize = samplesPerBlockExpected;
    spec.numChannels = 2;

    eqChain.prepare(spec);

    // Initialise filters (1 = flat / no changes).
    setEqLow(1.0f);
    setEqMid(1.0f);
    setEqHigh(1.0f);
}

// Process the audio through our resampling and EQ chain
void DJAudioPlayer::getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill)
{
    resampleSource.getNextAudioBlock(bufferToFill);

    juce::dsp::AudioBlock<float> block(*bufferToFill.buffer);

    auto subBlock = block.getSubsetChannelBlock(bufferToFill.startSample,
(size_t)bufferToFill.numSamples);

    juce::dsp::ProcessContextReplacing<float> context(subBlock);

    eqChain.process(context);
}

// Update the Low frequency shelf filter
void DJAudioPlayer::setEqLow(float gainLinear)
{
    auto coeffs =
juce::dsp::IIR::Coefficients<float>::makeLowShelf(currentSampleRate, 200.0f,
0.707f, gainLinear);
    *eqChain.get<0>().coefficients = *coeffs;
}

// Update the Mid frequency peak filter
void DJAudioPlayer::setEqMid(float gainLinear)
{
    auto coeffs =
juce::dsp::IIR::Coefficients<float>::makePeakFilter(currentSampleRate, 5000.0f,
0.707f, gainLinear);
    *eqChain.get<1>().coefficients = *coeffs;
}

// Update the High frequency shelf filter
void DJAudioPlayer::setEqHigh(float gainLinear)
{
    auto coeffs =
juce::dsp::IIR::Coefficients<float>::makeHighShelf(currentSampleRate, 12000.0f,
0.707f, gainLinear);
    *eqChain.get<2>().coefficients = *coeffs;
}

// Calculate the current BPM (assuming a base BPM if metadata is missing)
```

```
double DJAudioPlayer::getBpm()
{
    if (readerSource != nullptr && readerSource->getAudioFormatReader() !=
nullptr)
    {
        double baseBpm = 120.0;
        juce::String bpmMetadata = readerSource->getAudioFormatReader()-
>metadataValues.getValue("bpm", "");
        if (bpmMetadata.isNotEmpty())
        {
            baseBpm = bpmMetadata.getDoubleValue();
        }
        return baseBpm * resampleSource.getResamplingRatio();
    }
    return 0;
}
```

MainComponent.h

```
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"
#include "DJAudioPlayer.h"
#include "DeckGUI.h"
#include "PlaylistComponent.h"

class MainComponent   : public AudioAppComponent
{
public:
    /** Constructor: sets up the audio format manager, players, and the main
layout. */
    MainComponent();

    /** Destructor: shuts down the audio system and cleans up resources. */
    ~MainComponent();

    /** Prepares the audio channels and mixer for playback.
     * @param samplesPerBlockExpected The number of samples the audio device will
request.
     * @param sampleRate The output sample rate of the audio device.
     */
    void prepareToPlay (int samplesPerBlockExpected, double sampleRate) override;

    /** Combines the audio streams from both decks and sends them to the output.
     * @param bufferToFill Information about the audio buffer to be processed.
     */
    void getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill) override;

    /** Releases audio resources when they are no longer needed. */
    void releaseResources() override;
```

```
    /** Paints any background visuals or branding for the main application window.
     * @param g The graphics context used for drawing.
     */
    void paint (Graphics& g) override;

    /** Handles the responsive layout of the decks and the playlist. */
    void resized() override;

private:
    // Handles the different audio file formats
    AudioFormatManager formatManager;

    // Cache to store thumbnails so they don't have to be re-drawn constantly
    AudioThumbnailCache thumbCache{100};

    // Player and GUI for the first deck
    DJAudioPlayer player1{formatManager};
    DeckGUI deckGUI1{&player1, formatManager, thumbCache};

    // Player and GUI for the second deck
    DJAudioPlayer player2{formatManager};
    DeckGUI deckGUI2{&player2, formatManager, thumbCache};

    // Combines multiple audio sources into one output stream
    MixerAudioSource mixerSource;

    // The component that manages our track list and loading logic
    PlaylistComponent playlistComponent{&deckGUI1, &deckGUI2};

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (MainComponent)
};
```

MainComponent.cpp

```
#include "MainComponent.h"

MainComponent::MainComponent()
{
    // Make sure you set the size of the component after
    // you add any child components.
    setSize (1400, 900);

    // Colors from Decks.
    deckGUI1.setDeckColor(juce::Colours::darkmagenta);
    deckGUI2.setDeckColor(juce::Colours::cyan);

    // Some platforms require permissions to open input channels so request that
here
    if (RuntimePermissions::isRequired (RuntimePermissions::recordAudio)
        && ! RuntimePermissions::isGranted (RuntimePermissions::recordAudio))
    {
```

```
        RuntimePermissions::request (RuntimePermissions::recordAudio,
                                     [&] (bool granted) { if (granted)
setAudioChannels (2, 2); });
    }
    else
    {
        // Specify the number of input and output channels that we want to open
        setAudioChannels (0, 2);
    }

    // Making our GUI components visible on the screen
    addAndMakeVisible(deckGUI1);
    addAndMakeVisible(deckGUI2);
    addAndMakeVisible(playlistComponent);

    // Getting the format manager ready to handle common audio files
    formatManager.registerBasicFormats();
}

MainComponent::~MainComponent()
{
    // This shuts down the audio device and clears the audio source.
    shutdownAudio();
}

void MainComponent::prepareToPlay (int samplesPerBlockExpected, double sampleRate)
{
    // Setting up our players for the upcoming audio playback
    player1.prepareToPlay(samplesPerBlockExpected, sampleRate);
    player2.prepareToPlay(samplesPerBlockExpected, sampleRate);

    // Setting up the mixer and adding our decks as inputs
    mixerSource.prepareToPlay(samplesPerBlockExpected, sampleRate);

    mixerSource.addInputSource(&player1, false);
    mixerSource.addInputSource(&player2, false);

 }

void MainComponent::getNextAudioBlock (const AudioSourceChannelInfo& bufferToFill)
{
    // Fetching the combined audio from the mixer to play it out
    mixerSource.getNextAudioBlock(bufferToFill);
}

void MainComponent::releaseResources()
{
    player1.releaseResources();
    player2.releaseResources();
    mixerSource.releaseResources();
}

void MainComponent::paint (Graphics& g)
{
```

```
    // Component is opaque, so we must completely fill the background with a solid
colour.
    g.fillAll (getLookAndFeel().findColour (ResizableWindow::backgroundColourId));
}

void MainComponent::resized()
{
    // Divides screen.
    auto deckHeight = getHeight() * 0.6;

    // Decks upper part.
    deckGUI1.setBounds(0, 0, getWidth()/2, deckHeight);
    deckGUI2.setBounds(getWidth()/2, 0, getWidth()/2, deckHeight);

    // Playlist bottom part.
    playlistComponent.setBounds(0, deckHeight, getWidth(), getHeight() -
deckHeight);
}
```

Entendido. Vou aplicar a mesma técnica de proteção para garantir que você veja os crases triplos e a sintaxe Markdown corretamente para copiar.

Aqui estão as alterações para os arquivos WaveformDisplay:

WaveformDisplay.h

```
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"


class WaveformDisplay    : public Component,
                           public ChangeListener
{
public:
    /** Constructor: sets up the audio thumbnailer and registers the change
listener.
     * @param formatManagerToUse The manager that handles audio format reading.
     * @param cacheToUse The cache used to store and quickly retrieve thumbnails.
     */
    WaveformDisplay( AudioFormatManager &   formatManagerToUse,
                     AudioThumbnailCache &   cacheToUse );

    /** Destructor: cleans up the thumbnail resources when the display is
destroyed. */
    ~WaveformDisplay();

    /** Paints the visual representation of the audio waveform.
     * @param g The graphics context used for drawing the waveform and playhead.
     */
    void paint (Graphics& g) override;
```

```
    /** Handles the responsive layout of the waveform component. */
    void resized() override;

    /** Receives updates from the thumbnailer to redraw when the audio finishes
loading.
     * @param source The broadcaster that sent the change notification.
     */
    void changeListenerCallback (ChangeBroadcaster *source) override;

    /** Loads a specific audio file to be drawn as a waveform.
     * @param audioURL The URL of the audio file to process.
     */
    void loadURL(URL audioURL);

    /** set the relative position of the playhead*/
    void setPositionRelative(double pos);

    /** Changes the colour of the waveform lines for visual customisation.
     * @param newColour The new colour to be applied to the waveform.
     */
    void setWaveformColour(juce::Colour newColour);

private:
    // The core object that handles the audio visual data
    AudioThumbnail audioThumb;

    // Flag to check if we actually have a file to draw
    bool fileLoaded;

    // Current position of the playhead (0.0 to 1.0)
    double position;

    // The primary colour used to draw the waveform lines
    juce::Colour waveformColour = juce::Colours::orange;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (WaveformDisplay)
};
```

WaveformDisplay.cpp

```
#include "../JuceLibraryCode/JuceHeader.h"
#include "WaveformDisplay.h"

WaveformDisplay::WaveformDisplay(AudioFormatManager &   formatManagerToUse,
                                 AudioThumbnailCache &  cacheToUse) :
                                 audioThumb(1000, formatManagerToUse, cacheToUse),
                                 fileLoaded(false),
                                 position(0)

{
```

```
  // Registering the display to listen for changes in the audio thumbnail
  audioThumb.addChangeListener(this);
}

WaveformDisplay::~WaveformDisplay()
{
}

void WaveformDisplay::paint (Graphics& g)
{
    g.fillAll (getLookAndFeel().findColour (ResizableWindow::backgroundColourId));
// clear the background

    g.setColour (Colours::grey);
    g.drawRect (getLocalBounds(), 1);   // draw an outline around the component

    g.setColour (waveformColour);

    if(fileLoaded)
    {
      // Drawing the actual waveform using the thumbnail data
      audioThumb.drawChannel(g,
        getLocalBounds(),
        0,
        audioThumb.getTotalLength(),
        0,
        1.0f
      );

      // Drawing the playhead indicator over the waveform
      g.setColour(Colours::lightgreen);
      g.drawRect(position * getWidth(), 0, getWidth() / 20, getHeight());
    }
    else
    {
      g.setFont (20.0f);
      g.drawText ("File not loaded...", getLocalBounds(),
                  Justification::centred, true);   // draw some placeholder text

    }
}

void WaveformDisplay::loadURL(URL audioURL)
{
  // Clearing the previous thumbnail and loading the new audio source
  audioThumb.clear();
  fileLoaded  = audioThumb.setSource(new URLInputSource(audioURL));

  if (fileLoaded)
  {
    repaint();
  }
  else {
    std::cout << "Not loaded." << std::endl;
```

```
  }

}

void WaveformDisplay::changeListenerCallback (ChangeBroadcaster *source)
{
    // Redraw the component whenever the thumbnailer updates
    repaint();
}

void WaveformDisplay::setWaveformColour(juce::Colour newColour)
{
    waveformColour = newColour;
    repaint();
}

void WaveformDisplay::setPositionRelative(double pos)
{
  if (pos != position)
  {
    position = pos;
    repaint();
  }
}
```

PlaylistComponent.h

```
#pragma once

#include "../JuceLibraryCode/JuceHeader.h"
#include <vector>
#include <string>

// Forward declaration: to compiler "Know" that the class exists.
class DeckGUI;

// Structure to store data from each music.
struct Track {
    juce::String title;
    juce::URL url;
    juce::String format;
    double lengthInSecs;
};

/**
     * Structure to represent a single audio track in the library.
     * It stores the file name (title) and its memory location (URL).
     * It has inheritance from juce Component, TableListBoxModel and
Button::Listener.
 */
class PlaylistComponent : public juce::Component,
```

```
                            public juce::TableListBoxModel,
                            public juce::Button::Listener,
                            public juce::TextEditor::Listener
{
public:
    PlaylistComponent(DeckGUI* deck1, DeckGUI* deck2);
    ~PlaylistComponent();

    /** Paints the background of the table rows. */
    void paint (juce::Graphics&) override;

     /** Draws the text/data into each cell of the library table. */
    void resized() override;

    /** Returns the total number of tracks currently in the library. */
    int getNumRows() override;

    /** Draws the background for each row in the table. */
    void paintRowBackground (juce::Graphics&, int rowNumber, int width, int
height, bool rowIsSelected) override;

    /** Draws the specific data (text) into each cell of the table. */
    void paintCell (juce::Graphics&, int rowNumber, int columnId, int width, int
height, bool rowIsSelected) override;

    /** Insert the interactive components (buttons) in the cells. */
    juce::Component* refreshComponentForCell (int rowNumber, int columnId, bool
isRowSelected, juce::Component* existingComponentToUpdate) override;

    /** Handles button clicks, specifically the 'Import' action for R2A. */
    void buttonClicked(juce::Button* button) override;

    /** Stores the actual state from vector of tracks in disk. */
    void saveLibrary();

    /** Reads the disk and build the vector of tracks before initialization. */
    void loadLibrary();

    /** XXX */
    juce::TextEditor searchInput;

    /** XXX */
    std::vector<Track> allTracks;

    /** */
    void textEditorTextChanged(juce::TextEditor& editor) override;

private:
    juce::TableListBox tableComponent;
    juce::TextButton importButton{ "IMPORT TO LIBRARY" };

    // Vector that stores multiple Track objects for the library.
    std::vector<Track> tracks;
```

```
    std::unique_ptr<juce::FileChooser> fChooser;

    // Pointers to decks.
    DeckGUI* deck1;
    DeckGUI* deck2;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (PlaylistComponent)
};
```

PlaylistComponent.cpp

```
#include "PlaylistComponent.h"
#include "DeckGUI.h"

PlaylistComponent::PlaylistComponent(DeckGUI* _deck1, DeckGUI* _deck2):
deck1(_deck1), deck2(_deck2)
{
    // Configuration of columns from table.
    tableComponent.getHeader().addColumn("Track Title", 1, 300);
    tableComponent.getHeader().addColumn("Format", 2, 100);
    tableComponent.getHeader().addColumn("Duration", 3, 100);
    tableComponent.getHeader().addColumn("Load D1", 4, 80);
    tableComponent.getHeader().addColumn("Load D2", 5, 80);
    tableComponent.getHeader().addColumn("Remove", 6, 80);

    tableComponent.setModel(this);

    // Making the table and import button visible to the user
    addAndMakeVisible(tableComponent);
    addAndMakeVisible(importButton);

    // Setting up the listener for our import button
    importButton.addListener(this);

    // Adding the search bar and setting its placeholder text
    addAndMakeVisible(searchInput);
    searchInput.addListener(this);
    searchInput.setTextToShowWhenEmpty("Search tracks...",
juce::Colours::lightgrey);

    // Read saved file when opening the app.
    loadLibrary();
}

PlaylistComponent::~PlaylistComponent() {}

void PlaylistComponent::paint(juce::Graphics& g)
{
    // Filling the background with the default window colour

g.fillAll(getLookAndFeel().findColour(juce::ResizableWindow::backgroundColourId));
```

```
}

void PlaylistComponent::resized()
{
    int toolBarHeight = 40;
    int padding = 5;

    // Positioning the top buttons and search input
    importButton.setBounds(padding, padding, (getWidth() / 4) - (padding * 2),
toolBarHeight - (padding * 2));
    searchInput.setBounds(getWidth() / 4, padding, (getWidth() / 4 * 3) - padding,
toolBarHeight - (padding * 2));

    // The table takes up the rest of the window space below the toolbar
    tableComponent.setBounds(0, toolBarHeight, getWidth(), getHeight() -
toolBarHeight);
}

// Returns the size of the vector (how many rows the table should draw).
int PlaylistComponent::getNumRows()
{
    return static_cast<int>(tracks.size());
}

// Draw the background of the lines (blue if selected, grey if not).
void PlaylistComponent::paintRowBackground(juce::Graphics& g, int rowNumber, int
width, int height, bool rowIsSelected)
{
    if (rowIsSelected) g.fillAll(juce::Colours::lightblue);
    else g.fillAll(juce::Colours::darkgrey);
}

// Draw the text in each cell based on the columnId.
void PlaylistComponent::paintCell(juce::Graphics& g, int rowNumber, int columnId,
int width, int height, bool rowIsSelected)
{
    if (rowNumber < getNumRows())
    {
        g.setColour(juce::Colours::white);

        if (columnId == 1) // Title.
            g.drawText(tracks[rowNumber].title, 2, 0, width - 4, height,
juce::Justification::centredLeft, true);

        if (columnId == 2) // Format.
            g.drawText(tracks[rowNumber].format, 2, 0, width - 4, height,
juce::Justification::centredLeft, true);

        if (columnId == 3) // Duration.
            g.drawText(juce::String( (int)tracks[rowNumber].lengthInSecs ) + "s",
2, 0, width - 4, height, juce::Justification::centredLeft, true);
    }
}
```

```
void PlaylistComponent::buttonClicked(juce::Button* button)
{
    if (button == &importButton)
    {
        // Define to open files and allow multiple files.
        auto fileChooserFlags = juce::FileBrowserComponent::canSelectFiles |

juce::FileBrowserComponent::canSelectMultipleItems;

        // Create the selector (using std::make_unique to manage memory).
        fChooser = std::make_unique<juce::FileChooser>("Select files to add to
library...",
                                                       juce::File{},
                                                       "*.mp3;*.wav;*.aif");

        // Assync form (modern JUCE).
        fChooser->launchAsync(fileChooserFlags, [this](const juce::FileChooser&
chooser)
        {
            auto results = chooser.getResults(); // Gets the list of selected
files

            for (const auto& file : results)
            {
                Track newTrack;
                newTrack.title = file.getFileName();
                newTrack.url = juce::URL{file};
                newTrack.format = file.getFileExtension();

                juce::AudioFormatManager formatManager;
                formatManager.registerBasicFormats();
                std::unique_ptr<juce::AudioFormatReader>
reader(formatManager.createReaderFor(file));

                if (reader != nullptr) {
                    newTrack.lengthInSecs = reader->lengthInSamples / reader-
>sampleRate;
                } else {
                    newTrack.lengthInSecs = 0;
                }

                tracks.push_back(newTrack);
            }

            // Music.
            allTracks = tracks;

            // Tells table to update the list in the screen.
            tableComponent.updateContent();

            // Persisting the updated track list to a file
            saveLibrary();
        });
    }
```

```
}

juce::Component* PlaylistComponent::refreshComponentForCell(int rowNumber, int
columnId, bool isRowSelected, juce::Component* existingComponentToUpdate)
{

    if (columnId == 4 || columnId == 5 || columnId == 6)
    {
        auto* btn = static_cast<juce::TextButton*>(existingComponentToUpdate);

        if (btn == nullptr)
        {
            // Defines the name for the button (columns 4, 5 and 6).
            juce::String btnName = (columnId == 4) ? "Deck 1" : (columnId == 5 ?
"Deck 2" : "Remove");
            btn = new juce::TextButton(btnName);
        }

        // Stores the actual row in the ID of the button for later reference.
        btn->setComponentID(juce::String(rowNumber));

        // Defines the action of the click in memory.
                btn->onClick = [this, btn, columnId]()
                {
                    int row = btn->getComponentID().getIntValue();

                    if (columnId == 4) {
                        // Send URL from music to deck 1.
                        deck1->loadFile(tracks[row].url);
                    } else if (columnId == 5) {
                        // Send URL from music to deck 2.
                        deck2->loadFile(tracks[row].url);

                    } else if (columnId == 6) {

                        // Using async call to safely remove tracks without
crashing the UI
                        juce::MessageManager::callAsync([this, row]() {
                            if (row < tracks.size()) {

                            // Deletes backup.
                            for (int i = 0; i < allTracks.size(); ++i) {
                                if (allTracks[i].url == tracks[row].url) {
                                    allTracks.erase(allTracks.begin() + i);
                                    break;
                                }
                            }
                            // Delete from screen.
                            tracks.erase(tracks.begin() + row);
                            tableComponent.updateContent();
                            saveLibrary();
                            }
                        });
                    }
```

```
                };
            return btn;
        }
        return nullptr;
    }

    void PlaylistComponent::saveLibrary()
    {
        // Creates (or locates) a file "OtoDecksLibrary.txt" in the documents
    directory of the OS.
        juce::File libraryFile =
    juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile("O
    toDecksLibrary.txt");

        // Opens the output flow of data.
        juce::FileOutputStream output(libraryFile);
        if (!output.openedOk()) return;

        output.setPosition(0);
        output.truncate();

        // Iterates through the vector and stores the path of each track.
        for (const auto& track : tracks)
        {
            // Collects the absolute path from Windows / Mac and adds a line break.
            juce::String filePath = track.url.getLocalFile().getFullPathName();
            output.writeText(filePath + "\n", false, false, nullptr);
        }
    }

    void PlaylistComponent::loadLibrary()
    {
        // Points to the same persistence file used for saving.
        juce::File libraryFile =
    juce::File::getSpecialLocation(juce::File::userDocumentsDirectory).getChildFile("O
    toDecksLibrary.txt");

        if (libraryFile.existsAsFile())
        {
            // Read all lines from file once.
            juce::StringArray lines;
            libraryFile.readLines(lines);

            // Build the Track objects and allocate them in memory.
            for (juce::String line : lines)
            {
                if (line.isNotEmpty())
                {
                    juce::File file{line};

                    if (file.exists()) {
                        Track newTrack;
                        newTrack.title = file.getFileName();
                        newTrack.url = juce::URL{file};
```

```
                    newTrack.format = file.getFileExtension();

                    juce::AudioFormatManager formatManager;
                    formatManager.registerBasicFormats();
                    std::unique_ptr<juce::AudioFormatReader>
reader(formatManager.createReaderFor(file));

                    if (reader != nullptr) {
                        newTrack.lengthInSecs = reader->lengthInSamples / reader-
>sampleRate;
                    } else {
                        newTrack.lengthInSecs = 0;
                    }

                    tracks.push_back(newTrack);
                }
            }
        }

        // Music.
        allTracks = tracks;

        // Updates table with new loaded data.
        tableComponent.updateContent();
    }
}

void PlaylistComponent::textEditorTextChanged(juce::TextEditor& editor)
{
    if (searchInput.getText().isEmpty()) {
        // Restores everything if the text is erased.
        tracks = allTracks;
    } else {
        tracks.clear();
        for (const auto& track : allTracks) {
            // Compares the track title with the search input.
            if (track.title.containsIgnoreCase(searchInput.getText())) {
                tracks.push_back(track);
            }
        }
    }
    // Refreshing the table content to show filtered results
    tableComponent.updateContent();
}
```